

capacity(), on the other hand, tells you the total amount of memory allocated to a StringBuffer.

Method reverse() simply reverses the order of the characters in the StringBuffer.

```
StringBuffer king;  
king = new StringBuffer( "murder" );  
king.reverse();  
{ king.toString() == "redrum" };
```

Method ensureCapacity() simply makes sure that your StringBuffer has at least the capacity you specify. This is primarily an issue of efficiency. One could say the argument of this method is something called "Minimum capacity."

```
ensureCapacity( int minimumCapacity );  
There is a twist on how you apply ensureCapacity()  
StringBuffer sb;
```

```
sb = new StringBuffer("Test");
```

First (Certification-type question) What is the current capacity of StringBuffer object sb?

20 = 16 + 4

Since we know sb has a current capacity of 20, let's try a few experiments:

```
StringBuffer sb;  
sb = new StringBuffer("Test");  
sb.ensureCapacity( 19 );
```

What would be the capacity of sb after this statement?

20 = 16 + 4

What would be the capacity of sb after this statement?

```
StringBuffer sb;  
sb = new StringBuffer("Test");  
sb.ensureCapacity( 21 );
```

If you asked for less than what was already allocated, nothing changes. However, if you asked to increase the allocation, it gives you the larger of two-times the original plus 2 OR your request.

42

Finally, let's test that:

```
StringBuffer sb;  
sb = new StringBuffer("Test");  
sb.ensureCapacity( 43 );
```

Since this request is larger than two-times the original allocation, plus